

PI - MPGS SouthEast Bank- Documentation

Document Number	OI-PI-20260326
Document status	IN-REVIEW
Document owner	@Shayan Rasheed
Solutions Architect	@Maqsood Ali @Owais Khalid @Saqlain Raza
Project Manager	N/A
Priority	HIGH
Region	Bangladesh
Operator	MPGS SouthEast Bank
Product	PAYIN

Note: We have not got the testing credentials for MPGS. Hence this document is created as per the API document provided by the bank.

1. Overview

Mastercard Payment Gateway Services (MPGS) Direct Payment allows the operator to collect card details on its own application or website and send payment details directly to the Mastercard gateway for processing. This model provides full control over the payment experience, supports real-time synchronous API responses, and enables advanced payment operations including Pay, Authorize, Capture, Refund, Void, and Retrieve.

Because card details are collected directly by the operator, this integration increases PCI DSS compliance scope and requires the operator to manage all associated security controls.

Supported Transaction Flows

1. **PAY** — Combined authorize and capture in one step.
2. **AUTHORIZE → CAPTURE** — Two-step reservation and settlement.
3. **VERIFY** — Zero-amount card validation before payment.
4. **REFUND** — Return funds to payer after settlement.

5. **VOID** — Cancel a transaction before bank processing.
6. **RETRIEVE** — Query transaction or order state.

Integration Path

1. Obtain merchant profile and API access from Southeast Bank / MPGS provider.
2. Confirm which transaction operations are enabled on your merchant profile.
3. Select the authentication method: API password or SSL certificate.
4. Verify gateway connectivity using the `/information` endpoint before building flows.
5. Build server-side request handling for all transaction creation and response parsing.
6. Implement initial transaction operations (PAY, AUTHORIZE, VERIFY) per use case.
7. Implement subsequent operations (CAPTURE, REFUND, VOID, RETRIEVE).
8. Complete testing, settlement validation, and live credential switch before go-live.

Security Requirement: All API requests must be made from the merchant's backend server. Never invoke any API operation directly from the payer's browser, except specific 3D Secure authentication requests if applicable. Store `{{apiPassword}}` and `{{merchantId}}` exclusively in server-side secret management. Never embed them in frontend code, hidden form fields, or version control.

2. Environments & Base URLs

All endpoint paths in this document are relative to the appropriate base URL. Replace `{{baseUrl}}` with the correct environment URL in all cURL examples.

Environment	Base URL	Notes
Test	<code>https://test-southeastbank.mtf.gateway.mastercard.com/api/rest/version/100</code>	Sandbox for integration validation.

Production	<code>https://{{productionHost}}/api/rest/version/100</code>	Live endpoint provided during go-live.
-------------------	--	--

Gateway Connectivity Check

Use this before building any payment flows to confirm the gateway is reachable and operational.

Method: GET

Endpoint: /information

Sample Request (cURL)

```
1 curl --request GET \
2   --url "{{baseUrl}}/information"
3
```

Expected Response

```
1 {"status":"OPERATING"}
2
```

If this response is not returned, there is a connectivity issue preventing requests from reaching the gateway. Do not proceed until this check passes.

3. Authentication

MPGS Direct Payment supports two authentication methods. Select one and configure it before sending any transaction request.

M = Mandatory | O = Optional | C = Conditional

3a. Password Authentication

Parameter	Location	Data Type	M/O/C	Sample Value	Description
<code>{{apiUsername}}</code>	Config	String	M	<code>merchant.TEST12345</code>	API username issued for gateway access.
<code>{{apiPassword}}</code>	Config	String	M	<code>{{apiPassword}}</code>	Password used for gateway

					authentication
					.

Sample Auth Header (cURL)

```
1 --user "{{apiUsername}}:{{apiPassword}}"
2
```

3b. SSL Certificate Authentication

If using SSL certificate authentication:

- Do **not** include the password authentication header.
- Always validate the SSL certificate path before connecting.
- Validating prevents malicious attacks and trust chain issues.

4. Request Construction Standards

All requests must use REST JSON format. Use the correct HTTP verb per operation type.

HTTP Method	Usage
PUT	Creates or updates a specific transaction resource.
POST	Creates a new collection (e.g., session, payment options inquiry).
GET	Retrieves information without modifying state.

Standard HTTP Headers

Parameter	Type	Data Type	M/O/C	Sample Value	Description
Content-Type	Header	String	M	application/json; charset=UTF-8	Request content type. UTF-8 supported; ISO-8859-1 required for card-

					processor fields.
Content-Length	Header	Integer	M	256	Body size in bytes.

URL Pattern

```
1 {{baseUrl}}/merchant/{{merchantId}}/order/{{orderId}}/transaction/{{transactionId}}
2
```

Identifier Guidelines

- Generate unique `{{orderId}}` and `{{transactionId}}` values per transaction.
- All subsequent operations on the same order must reuse the same `{{orderId}}`.
- Use natural keys from your system to support reconciliation and audit trails.

5. Initial Transaction Operations

Initial transactions open a new payment lifecycle for an order.

5a. Pay

The Pay operation combines authorization and capture into a single message. Funds are reserved and transferred in one step. Order certainty is automatically set to `FINAL`.

Method: `PUT`

Endpoint:

```
/merchant/{{merchantId}}/order/{{orderId}}/transaction/{{transactionId}}
}
```

Request Parameters

Parameter	Type	Data Type	M/O/C	Sample Value	Description
api0operation	Body	String	M	PAY	Must be <code>PAY</code> for this flow.
{{orderId}}	Path	String	M	ORDER-001	Unique order reference. Echoed in response and

					used to link subsequent operations.
<code>{{transactionId}}</code>	Path	String	M	TXN-001	Unique transaction identifier for this operation.
<code>order.amount</code>	Body	String	M	100.00	Transaction amount. Always calculate server-side immediately before sending.
<code>order.currency</code>	Body	String	M	BDT	ISO 4217 currency code.
<code>sourceOfFunds.type</code>	Body	String	M	CARD	Payment source type.
<code>sourceOfFunds.provided.card.number</code>	Body	String	M	5123456789 012346	Full card number provided by payer.
<code>sourceOfFunds.provided.card.expiry.month</code>	Body	String	M	05	Card expiry month (2-digit).
<code>sourceOfFunds.provided.card.expiry.year</code>	Body	String	M	29	Card expiry year (2-digit).

sourceOfFunds.provider.securityCode	Body	String	C	123	Card security code (CSC/CVV). Required if enabled on merchant profile.
order.statementDescriptor	Body	String	O	SoutheastBank	Text shown on payer statement to reduce chargebacks.

Sample Request (cURL)

```

1 curl --request PUT \
2   --url "
   {{baseUrl}}/merchant/{{merchantId}}/order/{{orderId}}/transaction/{{transactionId}}" \
3   --user "{{apiUsername}}:{{apiPassword}}" \
4   --header "Content-Type: application/json; charset=UTF-8" \
5   --data '{
6     "apiOperation": "PAY",
7     "order": {
8       "amount": "100.00",
9       "currency": "BDT"
10    },
11    "sourceOfFunds": {
12      "type": "CARD",
13      "provided": {
14        "card": {
15          "number": "{{cardNumber}}",
16          "expiry": {
17            "month": "{{expiryMonth}}",
18            "year": "{{expiryYear}}"
19          },
20          "securityCode": "{{securityCode}}"
21        }
22      }
23    }
24  }'
```

Sample Response

Success (HTTP 200):

```

1 {
2   "result": "SUCCESS",
3   "order": {
4     "id": "{{orderId}}",
5     "amount": "100.00",
6     "currency": "BDT",
7     "status": "CAPTURED"
```

```

8   },
9   "transaction": {
10    "id": "{{transactionId}}",
11    "type": "PAYMENT",
12    "authorizationCode": "{{authorizationCode}}"
13  },
14  "response": {
15    "gatewayCode": "APPROVED"
16  }
17 }
18

```

Failure (HTTP 400):

```

1  {
2    "result": "FAILURE",
3    "response": {
4      "gatewayCode": "DECLINED",
5      "gatewayRecommendation": "DO_NOT_RETRY"
6    }
7  }
8

```

5b. Authorize

The Authorize operation verifies payer card details, checks available funds, and reserves the requested amount without transferring funds. Reserved funds are typically held for 5–8 days depending on issuer and scheme rules.

Method: PUT

Endpoint:

```

/merchant/{{merchantId}}/order/{{orderId}}/transaction/{{transactionId}}
}

```

apiOperation: AUTHORIZE

Order Certainty Options

Field Value	Description
FINAL	Full authorized amount expected to be captured within the mandated window (~7 days). May qualify for lower processing fees.
ESTIMATED	Authorized amount is an estimate. May incur higher processing fees.

Sample Request (cURL)

```

1 curl --request PUT \
2   --url "
   {{baseUrl}}/merchant/{{merchantId}}/order/{{orderId}}/transaction/{{tr

```



```

26  ansactionId}}" \
3    --user "{{apiUsername}}:{{apiPassword}}" \
4    --header "Content-Type: application/json; charset=UTF-8" \
5    --data '{
6      "apiOperation": "AUTHORIZE",
7      "order": {
8        "amount": "100.00",
9        "currency": "BDT",
10       "certainty": "FINAL"
11     },
12     "sourceOfFunds": {
13       "type": "CARD",
14       "provided": {
15         "card": {
16           "number": "{{cardNumber}}",
17           "expiry": {
18             "month": "{{expiryMonth}}",
19             "year": "{{expiryYear}}"
20           },
21           "securityCode": "{{securityCode}}"
22         }
23       }
24     }
25   }'
26

```

Troubleshooting — Authorize

- Unused authorization: submit VOID with `transaction.targetTransactionId` to release funds.
- Partial capture remaining: use VOID on outstanding authorization if no further capture is needed.
- Expired authorization: the gateway will auto-void and reject any Capture attempts after expiry.

5c. Verify

The Verify operation validates payer account details using a zero-amount verification flow before executing a Pay or Authorize. Use this when your business flow requires account confirmation first.

Method: PUT

Endpoint:

```

/merchant/{{merchantId}}/order/{{orderId}}/transaction/{{transactionId}}
}

```

apiOperation: VERIFY

Sample Request (cURL)

```

1 curl --request PUT \
2   --url "
   {{baseUrl}}/merchant/{{merchantId}}/order/{{orderId}}/transaction/{{transactionId}}" \
3   --user "{{apiUsername}}:{{apiPassword}}" \
4   --header "Content-Type: application/json; charset=UTF-8" \

```

```

5  --data '{
6    "apiOperation": "VERIFY",
7    "sourceOfFunds": {
8      "type": "CARD",
9      "provided": {
10       "card": {
11         "number": "{{cardNumber}}",
12         "expiry": {
13           "month": "{{expiryMonth}}",
14           "year": "{{expiryYear}}"
15         }
16       }
17     }
18   }'
19
20

```

6. Subsequent Operations

Subsequent operations act on an order that already has an initial transaction. The gateway stores initial order and card data, so only operation-specific fields need to be provided. From API v15 onward, do not include card details in CAPTURE or REFUND requests.

6a. Capture

Capture transfers funds using a prior AUTHORIZE transaction.

Method: PUT

Endpoint:

```
/merchant/{{merchantId}}/order/{{orderId}}/transaction/{{transactionId}}
}
```

apiOperation: CAPTURE

Request Parameters

Parameter	Type	Data Type	M/O/C	Sample Value	Description
apiOperation	Body	String	M	CAPTURE	Must be CAPTURE .
transaction.targetTransactionId	Body	String	M	TXN-001	Transaction ID of the original AUTHORIZE.

<code>order.amount</code>	Body	String	M	<code>100.00</code>	Capture amount. May be less than authorized amount for partial captures.
<code>order.currency</code>	Body	String	M	<code>BDT</code>	Must match the original authorization currency.

Sample Request (cURL)

```

1 curl --request PUT \
2   --url "{{baseUrl}}/merchant/{{merchantId}}/order/{{orderId}}/transaction/{{captureTransactionId}}" \
3   --user "{{apiUsername}}:{{apiPassword}}" \
4   --header "Content-Type: application/json; charset=UTF-8" \
5   --data '{
6     "apiOperation": "CAPTURE",
7     "transaction": {
8       "targetTransactionId": "{{authTransactionId}}"
9     },
10    "order": {
11      "amount": "100.00",
12      "currency": "BDT"
13    }
14  }'
```

6b. Refund

Refund credits funds back to the payer for a completed PAY or CAPTURE transaction.

Method: PUT

Endpoint:

`/merchant/{{merchantId}}/order/{{orderId}}/transaction/{{refundTransactionId}}`

apiOperation: REFUND

Notes

- Cannot exceed the total captured amount unless excessive refund privilege is enabled.
- Do not include card details in refund requests (API v15+).

- Full and partial refunds are supported.

Sample Request (cURL)

```
1 curl --request PUT \  
2   --url "  
   {{baseUrl}}/merchant/{{merchantId}}/order/{{orderId}}/transaction/{{refundTransactionId}}" \  
3   --user "{{apiUsername}}:{{apiPassword}}" \  
4   --header "Content-Type: application/json; charset=UTF-8" \  
5   --data '{  
6     "apiOperation": "REFUND",  
7     "order": {  
8       "amount": "100.00",  
9       "currency": "BDT"  
10    }  
11  }'
```

6c. Void

Void immediately reverses a transaction that has not yet been sent for bank processing.

If the transaction has already been processed by the bank, use REFUND instead.

Method: PUT

Endpoint:

/merchant/{{merchantId}}/order/{{orderId}}/transaction/{{voidTransactionId}}

apiOperation: VOID

Sample Request (cURL)

```
1 curl --request PUT \  
2   --url "  
   {{baseUrl}}/merchant/{{merchantId}}/order/{{orderId}}/transaction/{{voidTransactionId}}" \  
3   --user "{{apiUsername}}:{{apiPassword}}" \  
4   --header "Content-Type: application/json; charset=UTF-8" \  
5   --data '{  
6     "apiOperation": "VOID",  
7     "transaction": {  
8       "targetTransactionId": "{{targetTransactionId}}"  
9     }  
10  }'
```

6d. Update Authorization

Extends the validity period of an existing authorization. Optionally increases or decreases the reserved amount. Requires the Update Authorization privilege on the merchant profile.

6e. Referral

Resubmits a referred transaction after obtaining an authorization code from the issuer following a “Refer to Issuer” response.

7. Retrieval Operations

Retrieval operations fetch data without impacting the order lifecycle.

7a. Retrieve Transaction

Method: GET

Endpoint:

/merchant/{merchantId}/order/{orderId}/transaction/{transactionId}

Sample Request (cURL)

```
1 curl --request GET \  
2   --url "  
   {{baseUrl}}/merchant/{merchantId}/order/{orderId}/transaction/{tra  
   nsactionId}" \  
3   --user "{{apiUsername}}:{{apiPassword}}"
```

7b. Retrieve Order

Returns order-level summary and all associated transactions.

Method: GET

Endpoint: /merchant/{merchantId}/order/{orderId}

Sample Request (cURL)

```
1 curl --request GET \  
2   --url "{{baseUrl}}/merchant/{merchantId}/order/{orderId}" \  
3   --user "{{apiUsername}}:{{apiPassword}}"
```

8. Payment Options Inquiry

Use PAYMENT OPTIONS INQUIRY to retrieve payment options configured on the merchant account, including supported currencies, payment types, and card prefixes.

Method: POST

Endpoint: /merchant/{merchantId}/paymentOptionsInquiry

Customization Filters

Field	Sample Value	Description
order.currency	BDT	Restrict to a specific currency.
paymentType	CREDIT	Restrict to a payment method type.
sourceOfFunds.provided. card.prefix	512345	Restrict by card BIN prefix.
transactionSource	INTERNET	Restrict by transaction source.

Sample Request (cURL)

```
1 curl --request POST \  
2   --url "{{baseUrl}}/merchant/{{merchantId}}/paymentOptionsInquiry" \  
3   --user "{{apiUsername}}:{{apiPassword}}" \  
4   --header "Content-Type: application/json; charset=UTF-8" \  
5   --data '{  
6     "order": {  
7       "currency": "BDT"  
8     },  
9     "paymentType": "CREDIT"  
10  }'  
11
```

9. Multiple Payment Sources

MPGS can process payment details from multiple sources in a single request.

Source	Priority
API request fields	1 — Highest
Session identifier	2
Card token	3 — Lowest

When the same field is provided in multiple sources, the highest-priority source is used.

10. Payment Enhancements

Enhancement	How to Implement
-------------	------------------

Statement descriptor	Include <code>order.statementDescriptor</code> to reduce chargebacks.
Tokenization	Store card details with payer consent. Use token in subsequent transactions.
3D Secure	Provide purchase history data to increase frictionless authentication.
Level 2 / Level 3 data	Use <code>order.item[n]</code> objects for corporate and purchasing cards.
Sector-specific data	Use <code>airline</code> , <code>cruise</code> , or <code>healthcare</code> objects for eligible industries.

11. Response Interpretation

Field	Meaning
<code>result = SUCCESS</code>	Transaction processed successfully.
<code>result = FAILURE</code>	Inspect <code>response.gatewayCode</code> for further details.
<code>response.gatewayCode = APPROVED</code>	Transaction approved by issuer.
<code>response.gatewayCode = DECLINED</code>	Transaction declined. Check <code>gatewayRecommendation</code> .

Always validate that the returned `order.amount` and `order.currency` match what was submitted.

12. Error Handling & Troubleshooting

Scenario	Resolution
No gateway response received	Wait 60 seconds and resubmit the identical request. Gateway deduplicates; no duplicate charges.

Transaction not approved	Inspect <code>result</code> and <code>response.gatewayCode</code> for decline reason.
Operation rejected by gateway	Verify the operation privilege is enabled on <code>{{merchantId}}</code> .
CAPTURE or REFUND rejected	Confirm card details were not included in the request (API v15+).
Connectivity failure	Hit <code>/information</code> endpoint — expect <code>{"status":"OPERATING"}</code> .
Live failure after successful test	Confirm <code>{{merchantId}}</code> and <code>{{apiPassword}}</code> were updated to live values.
Invalid acquirer functionality	Contact MPGS provider to confirm the acquirer supports the requested operation.

13. Testing Checklist

#	Test Case	Expected Outcome
1	Gateway connectivity check	<code>{"status":"OPERATING"}</code> returned.
2	Authentication validation	Credentials accepted; no 401 returned.
3	PAY — success	<code>result = SUCCESS</code> , <code>gatewayCode = APPROVED</code> .
4	PAY — declined	<code>result = FAILURE</code> , <code>gatewayCode = DECLINED</code> .
5	AUTHORIZE then CAPTURE	CAPTURE debits correct amount; currency matches.
6	REFUND — full	Full amount refunded to payer.
7	REFUND — partial	Partial amount refunded correctly.

8	VOID	Transaction reversed before bank processing.
9	RETRIEVE TRANSACTION	Correct state returned for <code>{{transactionId}}</code> .
10	RETRIEVE ORDER	All transactions listed under <code>{{orderId}}</code> .
11	No-response retry	Identical request resent after 60s; no duplicate charge.
12	Secure credential check	<code>{{apiPassword}}</code> not present in frontend or logs.

14. Go Live Checklist

- All test cases from Section 13 passed.
- Merchant profile reviewed for all required operation privileges.
- Live `{{merchantId}}` obtained (no TEST prefix).
- Live `{{apiPassword}}` stored in server-side secret manager.
- `{{baseUrl}}` updated to production endpoint.
- Live test transactions processed with a real card.
- Settlement confirmed into the expected bank account.
- TLS certificate validated on live production endpoint.
- Error handling and retry logic verified in production environment.

Appendix — Placeholder Reference

Placeholder	Description
<code>{{baseUrl}}</code>	MPGS gateway base URL (test or production).
<code>{{merchantId}}</code>	Merchant ID issued by Southeast Bank / MPGS.
<code>{{apiUsername}}</code>	API username (typically <code>merchant.{{merchantId}}</code>).

<code>{{apiPassword}}</code>	API password stored in server-side secret manager.
<code>{{orderId}}</code>	Unique order identifier generated per order.
<code>{{transactionId}}</code>	Unique transaction identifier per operation.
<code>{{captureTransactionId}}</code>	New transaction ID used for the CAPTURE operation.
<code>{{refundTransactionId}}</code>	New transaction ID used for the REFUND operation.
<code>{{voidTransactionId}}</code>	New transaction ID used for the VOID operation.
<code>{{authTransactionId}}</code>	Transaction ID of the original AUTHORIZE to reference.
<code>{{targetTransactionId}}</code>	Transaction ID being targeted for VOID.
<code>{{cardNumber}}</code>	Full card number from payer.
<code>{{expiryMonth}}</code>	Card expiry month (2-digit).
<code>{{expiryYear}}</code>	Card expiry year (2-digit).
<code>{{securityCode}}</code>	Card CVV / CSC from payer.
<code>{{authorizationCode}}</code>	Authorization code returned in AUTHORIZE response.
<code>{{productionHost}}</code>	Live gateway hostname provided at go-live.